

Reddit Clone — Design Spec

Date: 2026-04-07

Course: Internet Programming Spring 2026

Stack: MERN + Socket.io + Firebase + Cloudinary + Grok AI

1. Project Overview

A full Reddit replica built with the MERN stack. Covers all 10 basic requirements, AI post summarization, real-time chat, browser push notifications, mobile responsiveness, and 8 extra features. Target: maximum team grade (80% cap).

Team: 6 people — 3 backend, 3 frontend (all work across all features)

Timeline: ~1 month

2. Features Scope

Basic Requirements

1. Account creation & login (OTP-based)
2. View/Edit user profile
3. Create community
4. Join/Leave community
5. Create post
6. View community posts
7. Home feed (personalized)
8. Upvote/Downvote posts
9. Comment on posts
10. Search communities & users

Graded Extras

- AI post summarization via Grok (10%)
- Real-time chat — Socket.io (bonus +5%)
- Deployment — Render + Vercel (bonus +5%)

Extra Features (+2% each)

- Saved/bookmarked posts
 - Post flairs/tags
 - Dark mode
 - Sort comments (Best/New/Top)
 - Image posts (Cloudinary)
 - Nested comments (1 level deep)
 - User karma (post + comment karma tracked separately)
 - Notifications (in-app via Socket.io + off-app via Firebase FCM)
-

3. Architecture

Approach: Feature-based modular monorepo. Both backend and frontend organized by feature, enabling 6 people to work in parallel with minimal merge conflicts.

Repo Structure

```
reddit-clone/
├── backend/
│   ├── src/
│   │   ├── features/
│   │   │   ├── auth/          # routes, controller, model, OTP logic
│   │   │   ├── users/        # profile view/edit, search
│   │   │   ├── communities/   # CRUD, join/leave, search
│   │   │   ├── posts/        # CRUD, upvote/downvote, feed, flairs
│   │   │   ├── comments/     # CRUD, upvote, nested replies
│   │   │   ├── chat/         # Socket.io rooms & messages
│   │   │   └── ai/           # Grok summarization endpoint
│   │   ├── middleware/       # auth (JWT), error handler, upload (Cloudinary)
│   │   ├── config/          # DB, env, cloudinary, socket, firebase setup
│   │   └── server.js
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   ├── communities/
│   │   │   ├── posts/
│   │   │   ├── comments/
│   │   │   ├── chat/
│   │   │   └── users/
│   │   ├── pages/           # route-level components
│   │   ├── components/     # shared UI (Navbar, PostCard, Modal, etc.)
│   │   ├── hooks/          # shared custom hooks
│   │   ├── context/        # AuthContext, ThemeContext
│   │   ├── services/       # axios instance + base API calls
│   │   ├── App.jsx
│   └── package.json
├── .gitignore
└── README.md
```

Each feature folder (backend) contains: `routes.js` , `controller.js` , `model.js`

Each feature folder (frontend) contains: `components/` , `hooks/` , `api.js`

4. Git Workflow

Branches:

- `main` — production-ready, protected (no direct pushes)
- `dev` — integration branch, all features merge here
- `feature/<name>` — one branch per feature (e.g. `feature/auth` , `feature/posts`)
- `fix/<name>` — bug fixes (e.g. `fix/otp-expiry`)

Flow:

1. Branch off `dev` → work on `feature/<name>`
2. Open PR into `dev` when done
3. At least 1 teammate reviews and approves
4. Merge into `dev`
5. Stable milestone → PR from `dev` → `main`

Rules:

- Never push directly to `main` or `dev`
 - Branch names: lowercase with hyphens
 - Commit prefixes: `feat:`, `fix:`, `chore:`, `docs:`
 - Never commit `.env` — use `.env.example`
-

5. Authentication

Method: OTP-based (no passwords)

Registration Flow (6 steps)

1. Enter email → backend sends OTP via Nodemailer (Gmail SMTP)
2. Enter OTP → verified
3. Enter username (real-time availability check via debounced API call)
4. Choose gender (optional, skippable)
5. Select interests (predefined category grid)
6. Select tags (predefined topic chips under each interest) → "Start exploring Reddit"

UI: centered modal-style card, progress indicator at top, Reddit logo above.

Login Flow

1. Enter email → OTP sent
2. Enter OTP → JWT returned

OTP Rules

- 6-digit code, hashed with `bcrypt` before storing
- 10-minute expiry
- Wrong OTP: show error + remaining attempts (no lockout)
- Expired OTP: "Resend OTP" button appears
- Rate limited: 5 OTP requests per 15 min per IP

JWT

- Stored in `localStorage`
 - Sent as `Authorization: Bearer <token>`
 - 7-day expiry
 - Protected frontend routes redirect to `/login` if no token
 - `401` response → auto logout
-

6. Database Schema

Users

```
{
  "_id": ObjectId,
  "username": "String (unique)",
  "email": "String (unique)",
  "bio": "String",
  "profilePicture": "String (Cloudinary URL)",
  "gender": "String",
  "interests": ["String"],
  "tags": ["String"],
  "otp": "String (hashed)",
  "otpExpiry": "Date",
  "verified": "Boolean",
  "postKarma": "Number",
  "commentKarma": "Number",
  "savedPosts": ["ObjectId (ref: Posts)"],
  "fcmToken": "String",
  "createdAt": "Date"
}
```

Communities

```
{
  "_id": ObjectId,
  "name": "String (unique)",
  "description": "String",
  "rules": "String",
  "creator": "ObjectId (ref: Users)",
  "members": ["ObjectId (ref: Users)"],
  "memberCount": "Number",
  "banner": "String (Cloudinary URL)",
  "icon": "String (Cloudinary URL)",
  "flairs": ["String"],
  "createdAt": "Date"
}
```

Posts

```
{
  "_id": ObjectId,
  "title": "String",
  "content": "String",
  "image": "String (Cloudinary URL, optional)",
  "author": "ObjectId (ref: Users)",
  "community": "ObjectId (ref: Communities)",
  "flair": "String",
  "tags": ["String"],
  "upvotes": "Number",
  "downvotes": "Number",
  "upvoters": ["ObjectId (ref: Users)"],
}
```

```
"downvoters": ["ObjectId (ref: Users)"],
"commentCount": "Number",
"createdAt": "Date"
}
```

Comments

```
{
  "_id": ObjectId,
  "content": "String",
  "author": "ObjectId (ref: Users)",
  "post": "ObjectId (ref: Posts)",
  "parentComment": "ObjectId (ref: Comments, null if top-level)",
  "upvotes": "Number",
  "upvoters": ["ObjectId (ref: Users)"],
  "createdAt": "Date"
}
```

Messages (Chat)

```
{
  "_id": ObjectId,
  "sender": "ObjectId (ref: Users)",
  "receiver": "ObjectId (ref: Users)",
  "content": "String",
  "read": "Boolean",
  "createdAt": "Date"
}
```

Notifications

```
{
  "_id": ObjectId,
  "user": "ObjectId (ref: Users)",
  "type": "String (upvote | comment | reply)",
  "post": "ObjectId (ref: Posts)",
  "triggeredBy": "ObjectId (ref: Users)",
  "read": "Boolean",
  "createdAt": "Date"
}
```

7. API Endpoints

Auth

Method	Endpoint	Description
--------	----------	-------------

POST	/api/auth/send-otp	Send OTP to email
POST	/api/auth/verify-otp	Verify OTP, return JWT
POST	/api/auth/complete-profile	Set username, gender, interests, tags
GET	/api/auth/me	Get current user (JWT)
POST	/api/auth/logout	Clear session

Users

Method	Endpoint	Description
GET	/api/users/:id	View profile
PUT	/api/users/:id	Edit bio, profilePicture
GET	/api/users/search?q=	Search users
GET	/api/users/:id/saved	Get saved posts
POST	/api/users/:id/save/:postId	Save a post
DELETE	/api/users/:id/save/:postId	Unsave a post

Communities

Method	Endpoint	Description
POST	/api/communities	Create community
GET	/api/communities/:id	View community
GET	/api/communities	List all communities
GET	/api/communities/search?q=	Search communities
POST	/api/communities/:id/join	Join community
POST	/api/communities/:id/leave	Leave community

Posts

Method	Endpoint	Description
POST	/api/posts	Create post (with optional image)
GET	/api/posts/:id	View post + comments
GET	/api/communities/:id/posts?sort=	Community posts
GET	/api/feed?sort=	Personalized feed
DELETE	/api/posts/:id	Delete post
POST	/api/posts/:id/upvote	Upvote post

POST	/api/posts/:id/downvote	Downvote post
------	-------------------------	---------------

Comments

Method	Endpoint	Description
POST	/api/comments	Create comment (parentComment optional)
DELETE	/api/comments/:id	Delete comment
POST	/api/comments/:id/upvote	Upvote comment

AI

Method	Endpoint	Description
POST	/api/ai/summarize/:postId	Summarize post + top 10 comments via Grok

Notifications

Method	Endpoint	Description
GET	/api/notifications	Get user notifications
PUT	/api/notifications/:id/read	Mark as read

8. Frontend Pages & Routing

/	# Home feed
/r/popular	# Popular posts
/r/all	# All posts
/r/:name	# Community page
/r/:name/submit	# Create post
/post/:id	# Post detail + comments
/u/:username	# User profile
/u/:username/saved	# Saved posts
/search?q=&type=	# Search results
/settings	# Account settings
/notifications	# Notifications page
/chat	# Chat inbox
/chat/:userId	# Chat thread
/login	# Login
/register	# Multi-step onboarding

9. UI Design (Reddit Replica)

Navbar

- **Left:** Reddit alien logo + "reddit" wordmark
- **Next:** "Home ▾" dropdown (Home / Popular / All)

- **Center:** Full-width search bar — placeholder "Search Reddit", results filtered by Posts / Communities / People
- **Right (logged out):** "Get App" | "Log In" | "Sign Up"
- **Right (logged in):** "⇒ Create" | 💬 chat icon | 🔔 bell (badge) | karma + avatar ▾

Left Sidebar

Top Navigation

[icon] Home ← active highlight
 [icon] Popular
 [icon] News
 [icon] Explore
 [icon] Start a community

RECENT

[icon] r/recentlyVisited (up to 5 most recently visited communities)

COMMUNITIES

Manage Communities
 [icon] r/communityName
 See more ▾

RESOURCES

[icon] About Reddit
 [icon] Advertise
 [icon] Developer Platform
 [icon] Reddit Pro [BETA]
 [icon] Help
 [icon] Blog
 [icon] Careers
 [icon] Press

Footer (tiny muted text)

Best of Reddit · Reddit Rules
 Privacy Policy · User Agreement
 Accessibility · Copyright © 2026

Post Card

▲ [community icon] r/community · Posted by u/name 2h

Post Title [thumbnail]

[Flair badge]

Text preview (2 lines, fades out)

💬 42 Comments ➦ Share ⭐ Save ... More

▼

- Left strip: community color
- Vote column: ▲ score ▼ (orange/blue on active)
- Sort tabs: Best | Hot | New | Top | Rising

Community Page

- Full-width banner + community icon overlapping bottom-left
- Name, member count, online count, Join/Leave button
- Two columns: posts (70%) + sidebar (30%)
- Sidebar: About, Rules, Moderators, Create Post button

Post Detail

- Vote column + full content
- AI Summarize button → modal with Grok summary
- Comment sort: Best | Top | New
- Comment tree: nested 1 level, collapsible [-]/[+]

User Profile

- Banner + avatar + username + post karma + comment karma + cake day
- Tabs: Overview | Posts | Comments | About
- Right sidebar: bio, karma breakdown (post karma + comment karma separately)

Create Post

- Tabs: Text | Image
- Community selector, Title (300 chars), content/image, flair, tags
- Save Draft | Post buttons

Chat

Conversations	Thread
u/user1 · 2h	[bubble] hey
last message...	[bubble] hi [you]
	[Type a message...][Send]

Color Scheme

	Light	Dark
Background:	#dae0e6	#1a1a1b
Cards:	#ffffff	#272729
Navbar:	#ffffff	#1a1a1b
Border:	#edeff1	#343536
Text:	#1c1c1c	#d7dadc
Muted:	#878a8c	#818384
Upvote:	#ff4500	#ff4500
Downvote:	#7193ff	#7193ff
Links:	#0079d3	#d7dadc

Font: IBM Plex Sans

10. Mobile Responsive Design

Breakpoints:

- Mobile: < 640px

- Tablet: 640px–1024px
- Desktop: > 1024px

Styling: Tailwind CSS with responsive prefixes (`sm:` , `md:` , `lg:`)

Mobile changes:

- Navbar: logo + search icon + bell + avatar only
- Bottom nav bar: 🏠 Home | 🔍 Search | ✎ Create | 🔔 | 💬
- Left sidebar: hidden → hamburger slides in from left
- Right sidebar: hidden → collapsed sections below posts
- Feed: full width single column
- Chat: conversations list = full screen, tap → full screen thread

11. Real-Time Features

Chat (Socket.io)

- Rooms named: `chat_userId1_userId2` (sorted IDs)
- Events: `join_chat` , `send_message` , `receive_message` , `typing` , `message_read`
- JWT validated on Socket.io connection

Notifications (Hybrid)

- **In-app:** Socket.io personal room `notifications_userId` — bell badge updates in real-time
- **Off-app:** Firebase Cloud Messaging (FCM) — browser push notification when tab is closed
- Permission requested once on first login
- FCM token saved to user DB record, refreshed silently on each login
- Triggers: upvote on your post, comment on your post, reply to your comment

12. External Integrations

Cloudinary

- Used for: post images, profile pictures, community banners/icons
- Upload flow: frontend → backend → Cloudinary → URL stored in DB
- API keys kept server-side only

Grok AI (xAI)

- OpenAI-compatible API via `openai` npm package (xAI base URL)
- Endpoint fetches post + top 10 comments → sends to Grok
- Prompt: "Summarize this Reddit post and its top comments in 3-5 sentences"
- Rate limited: 1 request per post per user per 10 minutes

Nodemailer

- Gmail SMTP with app password
- HTML OTP email template with 6-digit code + 10-min expiry warning

Firebase Admin SDK

- Backend: `firebase-admin` to send FCM push notifications
- Frontend: `firebase` SDK + `firebase-messaging-sw.js` service worker

13. Feed Personalization

Score formula:

```
score = upvotes
      + (matching interests: +20 per match)
      + (matching tags: +30 per match)
      + (joined community: +50)
      - (time decay: -1 per hour old)
```

Feed sources (dynamic mix):

- Joined communities (weighted +50) — grows as user joins more
- Posts matching interests/tags — always included for discovery
- New users: 100% interest/tag based from day 1 (never empty)
- Experienced users: shifts toward joined communities over time

Pagination: cursor-based, 20 posts per page

Sort options: Best (personalized) | Hot | New | Top | Rising

14. Error Handling

Backend

- Global error handler middleware, consistent response:

```
{ "success": false, "message": "Human-readable error", "code": "ERROR_CODE" }
```

- Input validation via `express-validator`
- Common codes: `INVALID_OTP` , `OTP_EXPIRED` , `USERNAME_TAKEN` , `NOT_MEMBER` , `UNAUTHORIZED` , `NOT_FOUND`
- Rate limiting: 5 OTP requests per 15 min per IP (`express-rate-limit`)

Frontend

- Axios interceptor catches all API errors globally
- `401` → auto logout + redirect to `/login`
- Toast notifications for user-facing errors
- Inline form validation (username availability: debounced API check)
- Loading states on all buttons (prevent double submit)

OTP

- Wrong OTP: "Incorrect code, try again" + remaining attempts shown
 - Expired OTP: "Resend OTP" button appears
 - No lockout — user can keep trying or resend anytime
-

15. Tech Stack Summary

Layer	Technology
-------	------------

Frontend	React 18, React Router, Tailwind CSS, Context API
Backend	Node.js, Express.js
Database	MongoDB + Mongoose
Auth	JWT + OTP via Nodemailer (Gmail SMTP)
Real-time	Socket.io
Push notifications	Firebase Cloud Messaging
Image storage	Cloudinary
AI	Grok API (xAI, OpenAI-compatible)
API docs	Postman
Deployment	Render (backend) + Vercel (frontend) + MongoDB Atlas